Authored by: Arpit Raj, LNMIIT Jaipur
Date: Wednesday, October 28, 2026
© 2026 Arpit Raj. All rights reserved.

C++ Advanced Mechanics

API Contracts, const Correctness & Initialization

API Contract Design in OOP

API design example and How interfaces are beneficial in product

1. make a `bird.h` header file

```cpp
#ifndef BIRD_H
#define BIRD_H
// abstract base class
class bird {
public:
    virtual void eat() = 0;
    virtual void sleep() = 0;
    virtual ~bird() { }
};

// class eagle : public bird
class eagle : public bird {
public:
    void eat() override {
        cout << "i eat meat ";
    }
    void sleep() override {
        cout << "i dont sleep ";
    }
};
// similarly make more child classes
```

2. now make a `main.cpp`

```
#include "bird.h"

void birdFun(bird * b) {
    b->eat();
    b->sleep();
}

// now in main
bird * birdptr = new eagle();
birdFun(birdptr);

// O/p -> i eat meat , i dont sleep
```

benefit:

1. If we used

```
void birdfun(eagle * e) {
    e->eat();
    e->sleep();
}
```

and if tommorrow we need pigeon activity, need to replace all eagles in code.
with this we can simply change just one line `bird * birdptr = new Pigeon()`
secondly if in future a new bird is added, we just add its class in `bird.h` header.

fun fact

```
int x = 2
int main() {
    int x = 20
    cout << x // -> 20
    cout << ::x // -> 2
}
```

The const keyword

- -> declares that a variable, function or object is immutable, value cant change after initialization.

declaring a function as `const` means that, we cant modify state of the object it is called on.
compilers store `const` variables in read-only memory, for faster access time.

Understanding lvalue vs rvalue

LVALUE VS RVALUE DEFINITIONS

- **lvalue** -> Locator value -> values having a specific memory location, can appear on both LHS or RHS of `=`.
  - Eg: `int x;`, `arr[0]` etc
- **rvalue** -> read values -> a temporary value that doesnt have specific mem locations, can appear only on RHS of `=`.
  - Eg: literals like `(67)`, temp results like `a+b;`, `int &a = b`

- `const int * a = 2` -> means that the data inside `a` cant change, but the pointer itself can be reassigned.
  - `*a = 20` -> error
  - but `int b = 10; a = &b;` is allowed and now `*a` will print `10`.
- `int * const a = 2` -> means that now pointer cant be changed but value can be changed.

© Arpit Raj, LNMIIT Jaipur

Pointer const Mechanics (Right-to-Left Rule)

- basically if we write `const` before `*` the content will be `const`
- if we want both of them to be constant then `const int * const a = new int(10)`

hence

RIGHT-TO-LEFT RULE BREAKDOWN

1. `const int* a = new int(2)`
   - **Right to left rule (const int * a):** `a` is a pointer to an `int` which is `const`, so data is locked but pointer is not.
2. `int *const a = new int(2)`
   - `a` is a `const` pointer to an `int`, so point cant change but data can.
3. `const int *const a = new int(2)`
   - `a` is a constant point to an `int` which is also constant.

Class-Level const & mutable

```
class abc {
public:
    int z;
    int getz() const {
        z=10; // -> error
        return z;
    }
};
```

adding the `const` ensures that members of class are not changed within the function.

- # Now let there be a outside function

```
void print(const abc &a) {
    // a.z = 10; -> error
}
```

If we write any non constant function of the objects class within print it will give error.
Eg

```
class abc {
public:
    void a() const { }
    void b() { }
};

void fun(const abc &obj) {
    obj.a() // allowed
    obj.b() // error
}
```

🚨 CRITICAL CONSTRAINT

doesnt matter if `b` is changing class member or not, **CONST reference can only call const function.**

💡 USING THE MUTABLE KEYWORD

- # If we want to change a variable `x` inside a const function
- use `mutable int x=1;`

```
void fun() const {
    x = 10;
} // -> is ALLOWED
```

i.e promise of const is bypassed via mutable. However dont use in real world, mutable was created for debugging purpose.

Initialization Lists

Old style constructor

```
int x
int *y
abc (int _x, int _y) {
    this->x = _x
    this->y = new int(_y);
}
```

New style constructor

```
int x
int *y
abc (int _x, int _y) : x(_x),
y(new int(_y)) {
}
```

Benefit of initializer list:
imagine this

```
const int x;
abc(int _x = 0) {
    this->x = _x
} // will give error, even if we remove = 0
```

but `abc(int _x = 0) : x(_x) { }` -> no error